BlinkChart Landing Page Implementation Guide

Step-by-step developer guide for Analytics, Cookie Banner, and Rate Limiting

Recommended implementation order

1) Analytics 2) Cookie banner 3) Rate limiting. This order is best because cookie consent should control analytics loading, and rate limiting mainly protects your public form endpoints.

1. Scope and objective

This guide is for the BlinkChart landing page and public waitlist flow. It gives the offshore team a practical implementation plan using Next.js and Vercel.

- Analytics: capture page views, CTA clicks, and waitlist conversions.
- Cookie banner: collect consent and prevent analytics from loading before acceptance.
- Rate limiting: protect public POST endpoints such as /api/waitlist from abuse.

2. Recommended stack

Area	Recommended choice	Why
Analytics	Google Analytics 4	Simple, widely used, and enough for landing-page conversion tracking.
Cookie consent	Lightweight custom banner	Fast to implement for Necessary + Analytics categories.
Rate limiting	Vercel Firewall/WAF + app-level API guard	Good protection for waitlist endpoints without overengineering.

3. Phase 1 - Analytics

Objective: track page views, CTA clicks, waitlist submit start, waitlist success, waitlist error, and important section views.

Team tasks

- Create a GA4 property for blinkchart.ai.
- Create a Web Data Stream and copy the measurement ID (G-XXXXXXXXXX).
- Add the value to local .env and to Vercel environment variables.
- Add an analytics gate so GA only loads after cookie consent is accepted.
- Track CTA and waitlist events from the UI.
- Validate in GA Realtime or DebugView.

Environment variable

```
NEXT_PUBLIC_GA_ID=G-XXXXXXXXXX
```

Install package

```
npm install @next/third-parties
```

File: components/AnalyticsGate.tsx

```
"use client";

import { useEffect, useState } from "react";
import { GoogleAnalytics } from "@next/third-parties/google";

export default function AnalyticsGate() {
const [enabled, setEnabled] = useState(false);

useEffect(() => {
const consent = window.localStorage.getItem("bc_cookie_consent");
if (consent === "accepted") setEnabled(true);

const onConsent = () => setEnabled(true);
window.addEventListener("bc-consent-accepted", onConsent);

return () => window.removeEventListener("bc-consent-accepted", onConsent);
}, []);

const gaId = process.env.NEXT_PUBLIC_GA_ID;
if (!enabled || !gaId) return null;

return <GoogleAnalytics gaId={gaId} />;
}
```

File: lib/analytics.ts

```
export type TrackEventArgs = {
action: string;
category?: string;
label?: string;
value?: number;
};

declare global {
interface Window {
gtag?: (...args: any[]) => void;
}
}

export function trackEvent({
action,
category,
label,
value,
}: TrackEventArgs) {
if (typeof window === "undefined") return;
if (!window.gtag) return;

window.gtag("event", action, {
event_category: category,
event_label: label,
value,
});
}
```

Recommended event names

- click_waitlist_cta
- click_hero_demo
- waitlist_submit_started
- waitlist_submit_success
- waitlist_submit_error
- page_view_pricing

Example CTA usage

```
import { trackEvent } from "@/lib/analytics";

<button
onClick={() =>
trackEvent({
action: "click_waitlist_cta",
category: "engagement",
label: "hero",
})
}
>
Join Waitlist
</button>
```

Analytics QA checklist

- NEXT_PUBLIC_GA_ID exists in local and Vercel.
- Analytics does not load before consent.
- CTA click event appears in GA Realtime.
- Waitlist success event appears in GA Realtime.
- Staging and production are both tested.

4. Phase 2 - Cookie banner

Objective: show a consent banner on first visit and block analytics unless the visitor accepts.

Team tasks

- Add a banner component.
- Store consent in localStorage.
- Support accepted, rejected, and unset states.
- Load analytics only after accepted.
- Add a footer link to reopen cookie settings.
- Create Privacy Policy and Cookies Policy pages.

File: components/CookieBanner.tsx

```
"use client";

import { useEffect, useState } from "react";

export default function CookieBanner() {
const [visible, setVisible] = useState(false);

useEffect(() => {
const consent = window.localStorage.getItem("bc_cookie_consent");
if (!consent) setVisible(true);
}, []);

const acceptCookies = () => {
window.localStorage.setItem("bc_cookie_consent", "accepted");
window.dispatchEvent(new Event("bc-consent-accepted"));
setVisible(false);
};

const rejectCookies = () => {
window.localStorage.setItem("bc_cookie_consent", "rejected");
setVisible(false);
};

if (!visible) return null;

return (
<div className="fixed bottom-0 inset-x-0 z-50 border-t bg-white p-4 shadow-lg">
<div className="mx-auto max-w-6xl flex flex-col gap-3 md:flex-row md:items-center
md:justify-between">
<p className="text-sm text-gray-700">
We use necessary cookies to run the site and optional analytics cookies
to understand usage and improve BlinkChart.
</p>
<div className="flex gap-2">
<button onClick={rejectCookies} className="rounded border px-4 py-2 text-sm">
Reject
</button>
<button onClick={acceptCookies} className="rounded bg-black px-4 py-2 text-sm text-white">
Accept
</button>
</div>
</div>
</div>
);
}
```

File: components/CookieSettingsLink.tsx

```
"use client";

export default function CookieSettingsLink() {
const reopenCookieBanner = () => {
localStorage.removeItem("bc_cookie_consent");
window.location.reload();
};

return (
<button onClick={reopenCookieBanner} className="underline text-sm">
Cookie Settings
</button>
);
}
```

Minimum policy pages

- app/privacy/page.tsx
- app/cookies/page.tsx

Minimum policy content

- What user data the waitlist form collects.
- Whether analytics is used.
- How users can reject or withdraw consent.
- How users can contact BlinkChart about privacy or data requests.

Cookie QA checklist

- Banner appears on first visit.
- Banner disappears after Accept or Reject.
- GA loads only after Accept.
- GA never loads after Reject.
- Cookie Settings link reopens consent choice.
- Privacy and Cookies pages are linked in the footer.

5. Phase 3 - Rate limiting

Objective: protect public form endpoints from abuse, spam, and bot traffic.

Endpoints to protect

- POST /api/waitlist
- POST /api/contact if present
- POST /api/demo-request if present

Team tasks

- Add app-level rate limiting in form routes.
- Return a clean 429 response for blocked requests.
- Add a hidden honeypot field in the form.
- Add server-side validation for required fields and email format.
- Enable Vercel Firewall/Bot Protection in log mode first.

Install package

```
npm install @vercel/firewall
```

File: app/api/waitlist/route.ts

```
import { NextRequest, NextResponse } from "next/server";
import { rateLimit } from "@vercel/firewall";

type WaitlistPayload = {
firstName?: string;
lastName?: string;
email?: string;
notes?: string;
website?: string; // honeypot
};

function isValidEmail(email: string) {
return /^[^\s@]+@[^\s@]+\.[^\s@]+$/.test(email);
}

export async function POST(req: NextRequest) {
const ip =
req.headers.get("x-forwarded-for")?.split(",")[0]?.trim() || "anonymous";

const decision = await rateLimit({
key: ip,
window: "10 m",
rate: 5,
});

if (!decision.ok) {
return NextResponse.json(
{ ok: false, error: "Too many requests. Please wait and try again." },
{ status: 429 }
);
}

let body: WaitlistPayload;
try {
body = await req.json();
} catch {
return NextResponse.json(
{ ok: false, error: "Invalid request body." },
{ status: 400 }
);
}

const firstName = body.firstName?.trim() || "";
const email = body.email?.trim() || "";
const website = body.website?.trim() || "";

if (website) return NextResponse.json({ ok: true }, { status: 200 });

if (!firstName || !email || !isValidEmail(email)) {
return NextResponse.json(
{ ok: false, error: "Please provide a valid name and email." },
{ status: 400 }
);
}

return NextResponse.json(
{ ok: true, message: "Thanks for joining the BlinkChart waitlist." },
{ status: 200 }
);
}
```

Honeypot field example

```
<input
type="text"
name="website"
tabIndex={-1}
autoComplete="off"
className="hidden"
aria-hidden="true"
/>
```

Suggested starting threshold

- 5 requests per 10 minutes per IP on the waitlist endpoint.
- Do not rate limit normal landing page GET requests.
- Tighten only after reviewing real traffic and spam patterns.

Rate limiting QA checklist

- The 6th rapid submit returns 429.
- Normal users can still submit the form.
- Malformed email returns 400.
- Honeypot submissions are silently ignored.
- Vercel Firewall logs are reviewed after deployment.

6. Recommended project structure

```
app/
api/
waitlist/
route.ts
cookies/
page.tsx
privacy/
page.tsx
layout.tsx
page.tsx

components/
AnalyticsGate.tsx
CookieBanner.tsx
CookieSettingsLink.tsx
HeroCTAButton.tsx
WaitlistForm.tsx

lib/
analytics.ts
```

7. Offshore team execution order

Sprint	Primary tasks	Acceptance criteria
Sprint 1	Add GA ID, AnalyticsGate, analytics helper, CTA events, waitlist events	Events visible in GA Realtime; analytics blocked before consent
Sprint 2	Build cookie banner, cookie settings link, privacy page, cookies page	Consent stored correctly; GA loads only after accept
Sprint 3	Protect /api/waitlist, add honeypot, add validation, enable Vercel bot protection	429 works, spam reduced, logs monitored

8. Definition of done

- Analytics tracks real page and CTA activity.
- Cookie banner appears on first visit.
- Analytics does not run before consent.
- Privacy and Cookies pages exist.
- Waitlist API is rate-limited.
- Honeypot and Firewall reduce bot spam.
- Production environment variables are configured.
- Staging and production are both tested.

Practical recommendation for BlinkChart right now

Keep it simple: GA4, a lightweight custom cookie banner, and rate limiting only on form APIs. Add heavier compliance tooling or CAPTCHA only if traffic or spam volume justifies it.